

Document-level Relation Extraction & Knowledge Graph

DocRED 기반 End-to-End Pipeline

어서5시조 | 이수민 김예슬 박재윤 박정현

프로젝트 목표



문서 수준 관계 추출 (DocRED)

단일 문장이 아닌 전체 문서를
대상으로 엔티티 간 관계를 추출



Multi-sentence 관계 이해

여러 문장에 걸친 정보를 종합하여
관계를 추론



Knowledge Graph 구축

추출된 관계를 그래프 형태로
저장 및 활용



End-to-End Pipeline 설계

입력 문서부터 KG 생성까지
일관된 시스템 구축

문제 정의 — 왜 어려운가?

01

Cross-sentence Reasoning

여러 문장에 걸쳐 정보를 연결

02

Multi-hop Inference

$A \rightarrow B, B \rightarrow C$ 를 통해 $A \rightarrow C$ 유추

03

Multi-label Classification

하나의 엔티티 쌍에 복수 관계 존재

04

Severe Class Imbalance

대부분이 'no relation' — 극심한 불균형

Sentence-level vs Document-level

항목	Sentence-level	Document-level
입력	한 문장	전체 문서
관계	직접 관계	간접 관계 포함
출력	단일 라벨	Multi-label
추론 범위	문장 내	문서 전체 (Cross-sentence)
난이도	보통	높음 (Multi-hop 필요)

DocRED는 문서 전체를 이해해야 한다는 점에서 훨씬 복잡한 문제

데이터 형태 확인

Data Format:

```
{
  "title": "문서 제목",
  "sents": [
    [word in sent 0],
    [word in sent 1]
  ],
  "vertexSet": [
    [{ "name": mention_name,
      "sent_id": which sentence,
      "pos": position in sentence,
      "type": NER_type },
     {another mention}, ...]
  ],
  "labels": [
    { "h": idx of head entity,
      "t": idx of tail entity,
      "r": relation,
      "evidence": [sent ids] }
  ]
}
```

필드	설명
sents	문서를 구성하는 문장별 토큰 리스트
vertexSet	entity별 mention 목록 (sent_id, pos, type 포함)
labels	head_idx, tail_idx, relation_id, 근거 문장 목록
evidence	해당 relation의 근거가 된 문장 인덱스 (DREAM용)

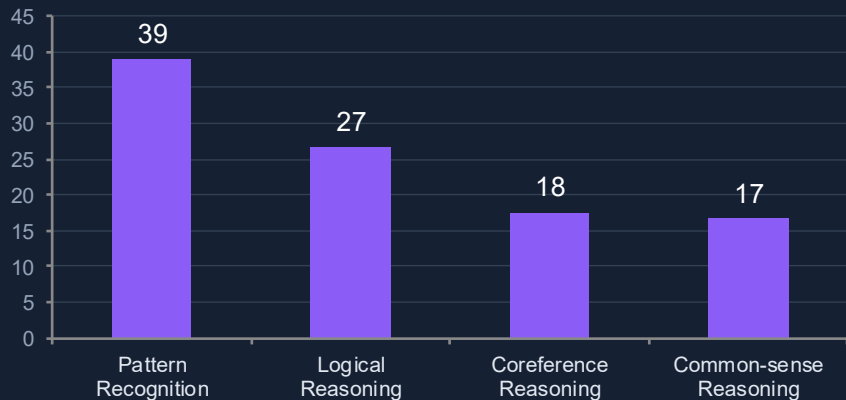
Relation 정보

- 총 97가지 relation: 96종 Wikidata property + 1 NA(관계 없음)
- 매핑 파일: data/meta/rel2id.json
- 예: "P17" → 국가, "P131" → 행정구역, "P710" → 참가자

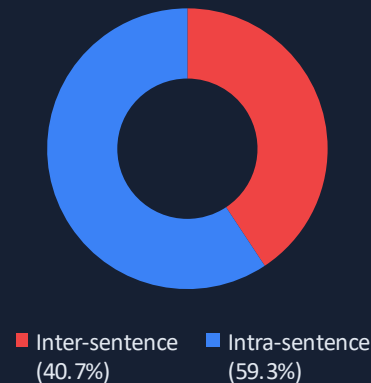
→ label 기준으로 F1 값을 측정

데이터 분석 ① 추론 유형 및 Evidence 분포

추론 유형별 비율 (DocRED 논문)



Inter vs Intra-sentence 관계 비율



40.7% Inter-sentence 관계

→ Cross-sentence GNN 도입 (Stage 3)

26.6% Logical Reasoning 필요

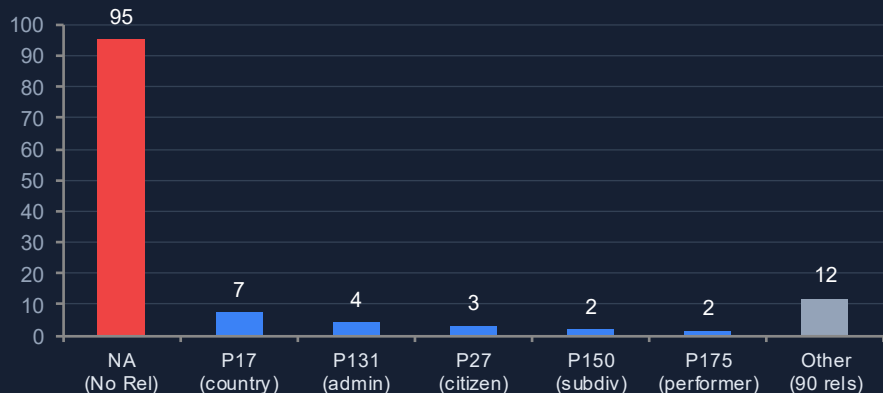
→ Multi-hop Graph 추론 (Stage 3/4)

46.4% 근거 2문장 이상 필요

→ Evidence Head 학습 (DREEM)

데이터 분석 ② 클래스 불균형 및 Entity 특성

Relation 분포 — 극심한 NA 불균형 (상대 비율)



DocRED 핵심 통계

문서 수	5,053 (Train 3,053 / Dev 1,000 / Test 1,000)
총 엔티티 수	132,375
총 관계 사실 수	56,354
관계 유형	96종 + 1 NA = 97
문서당 평균 엔티티	~19.5개
문서당 평균 후보 쌍	~390개 ($N \times (N-1)$)
평균 근거 문장 수	1.6문장/관계

95%+ NA (No Relation) 비율

→ NA weight 0.1 + Ranking Loss (Stage 1/2)

19.5 문서당 평균 엔티티 수

→ Entity Graph 구성 근거 (Stage 3/4)

~390 문서당 후보 쌍 ($N \times (N-1)$)

→ Adaptive Threshold 필수 (ATLOP)

구현 전략



Incremental Stacking

Stage 1 → 2 → 3 → 4 점진적 모듈 추가
이전 Stage 구조를 유지한 채 새 모듈만 결합
Stage 별 독립 평가로 기여도 측정



YAML 기반 Config 분기

stage1~4.yaml로 아키텍처 전환
`graph_encoder.architecture: gain | unet`
코드 수정 없이 설정만으로 실험 전환



Transfer Learning 전략

Stage 3/4 모두 Stage 2 체크포인트에서 시작
BERT + Relation Head 가중치를 그대로 전이
Graph Encoder만 새로 학습 → Ablation 가능



Differential Learning Rate

BERT Encoder: $2e-5$ (사전학습 보존)
Classifier / GNN: $1e-4$ (빠른 수렴)
Linear Warmup (6%) + Gradient Clipping (1.0)



Troubleshooting 기반 개선

v1 → v2: ATLOP Loss 구현 오류 수정
Eval 시 Adaptive TH 미적용 버그 해결
`pos_weight` 누락 → `BCEWithLogitsLoss` 수정

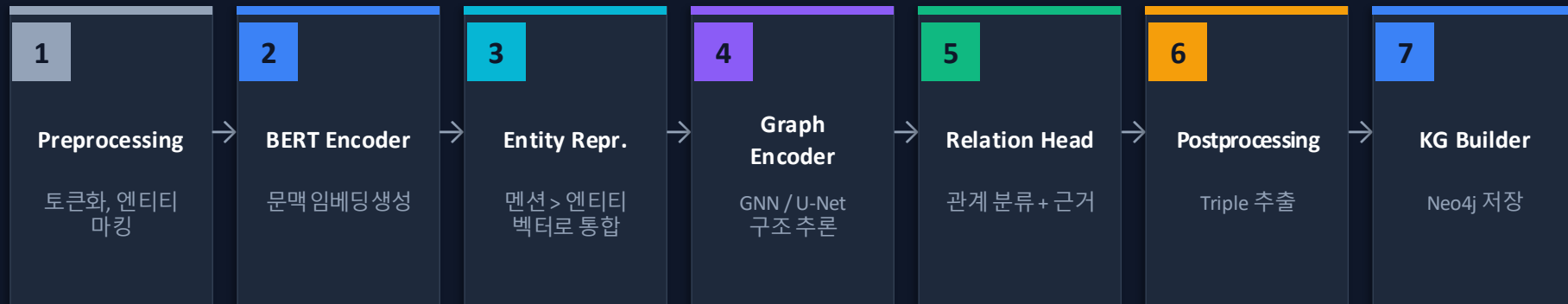


Class Imbalance 대응

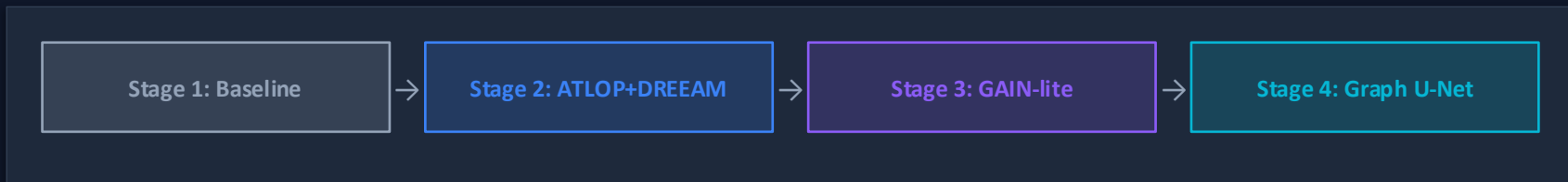
Stage 1: NA class weight 0.1로 BCE 보정
Stage 2+: Ranking Loss로 pos/neg margin 학습
Adaptive Threshold로 관계별 동적 임계값

전체 파이프라인 구조

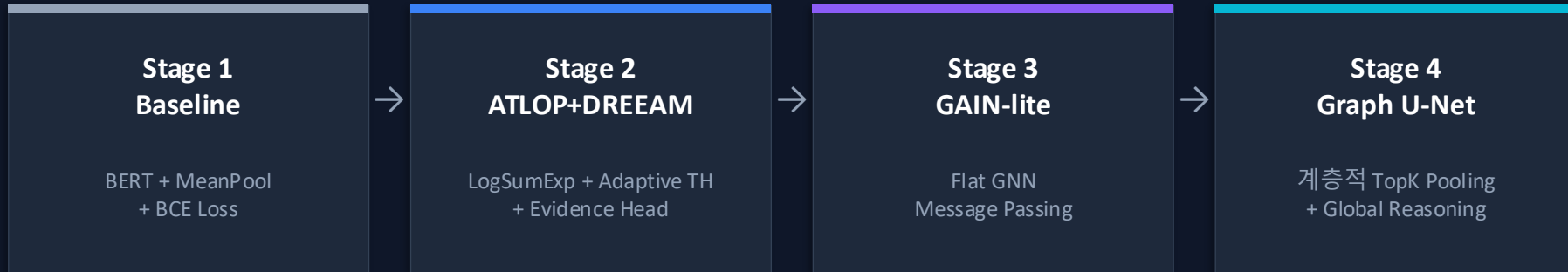
4-Stage Incremental Stacking



입력 문서 → 최종 Knowledge Graph까지 일관된 파이프라인



핵심 설계 전략 — Incremental Stacking



점진적 성능 향상

각 Stage에서 이전 결과를 유지하며
성능을 쌓아감



모듈 단위 검증

각 구성요소를 독립적으로 테스트
가능



디버깅 용이

문제 발생 시 어떤 모듈에서 발생했는지
추적 가능

* Stage 3과 Stage 4는 동일하게 Stage 2 체크포인트 기반 → Ablation 비교

Stage 1 — Baseline

양방향 언어 표현을 위한 트랜스포머 (Devlin et al. 2020)

1

BERT Encoder

사전학습된 BERT로
문서 토큰 임베딩 생성

2

Mean Pooling

멘션별 토큰을
평균 풀링하여 엔티티 표현

3

Bilinear Classifier

엔티티 쌍의 관계를
Bilinear 함수로 분류

4

BCE Loss

Multi-label 분류를 위한
Binary Cross-Entropy

Stage 1 핵심 설정

Pooling: mean | Classifier: bilinear MLP | Threshold: fixed (0.5) | Loss: weighted BCE (NA=0.1)
Optimizer: BERT LR 2e-5, Classifier LR 1e-4 | Epochs: 30 | Gradient Clipping: max_norm=1.0

이후 성능 비교의 기준점(Baseline) 역할

Stage 2 — ATLOP + DREEAM

적응형 문턱값 및 국소 문맥 풀링 (Zhou et al, 2021) | 근거 인지형 문서 레벨 관계 추출 (Ma et al, 2023)

LogSumExp Pooling

Attention score를 LogSumExp로 집계하여
중요한 멘션에 가중치 부여

Adaptive Threshold (ATLOP)

관계 별로 동적 임계값을 학습하여
더 정교한 Multi-label 분류 수행

Ranking Loss

Positive 관계와 Negative 관계 간 margin을 학습
 $\log(1+\sum \exp(\text{neg}-TH)) + \log(1+\sum \exp(TH-\text{pos}))$

Evidence Head (DREEAM)

모델이 어떤 문장을 근거로 사용했는지
명시적으로 학습 (KL Loss)

Stage 3 — GAIN-lite GNN

이중 그래프 기반 추론 네트워크 (Zeng et al, 2020)

Entity Graph + GNN Message Passing

Graph Neural Network를 도입하여 문서 전체 Entity 간 상호작용을 모델링합니다.

Entity Graph 구성:

- Co-occurrence Edge: 같은 문장에 등장하는 엔티티 간 연결
- Cross-sentence Edge: 인접 문장에 걸쳐 등장하는 엔티티 연결
- Self-loop: 자기 자신에 대한 정보 유지 연결

GCN/GAT 2-layer Message Passing + Residual + LayerNorm

Multi-hop Reasoning



$A \rightarrow B \rightarrow C \Rightarrow A \rightarrow C$ 추론

이를 통해 multi-hop 관계 추론이 가능해짐

Stage 4 — Graph U-Net

계층적 TopK 풀링 기반 전역 추론 (Gao & Ji, 2019)



핵심 메커니즘

GraphTopKPool: $\text{score} = \text{sigmoid}(\text{Linear}(x)) \rightarrow$ 상위 K개 노드 선별 + Gate 메커니즘으로 역전파 학습

Residual Connection: $\text{out} = \text{LayerNorm}(\text{entity_vectors} + h_{\text{out}}) \rightarrow$ Stage 2 벡터에 U-Net 델타값 추가

Ablation 목적: Stage 3 (Flat GNN) vs Stage 4 (U-Net) — 동일 Stage 2 체크포인트에서 비교

Stage 3 vs Stage 4 — Ablation 비교

구성 요소	Stage 3 (GAIN-lite)	Stage 4 (Graph U-Net)
그래프 구조	Flat 2-layer GNN	U-Net 계층 구조
노드 처리	전체 노드 균등 처리	중요 노드 선별 (TopK Pool)
추론 범위	1~2 hop 이웃	글로벌 Bottleneck 추론
Skip Connection	없음	있음 (로컬+글로벌 융합)
시작 체크포인트	Stage 2 best_model.pt	Stage 2 best_model.pt
모듈 파일	graph_encoder.py	structural_encoder.py

두 접근법 모두 Stage 2 기반으로 시작 → 순수하게 Graph Encoder 구조의 효과만 비교

Layer별 구조 요약

Preprocessing

토큰화, 엔티티 마킹, pair 생성



Encoder (BERT)

마지막 3개 layer 평균 → 문맥 임베딩



Entity Representation

Stage1: MeanPool / Stage2+: LogSumExp



Graph Layer

Stage3: GAIN-lite GNN / Stage4: Graph U-Net



Relation Head

ATLOP Bilinear + Adaptive TH + Evidence Head

Knowledge Graph 구축



Triple 생성

(head, relation, tail)
형태의 관계 트리플 추출



Neo4j 저장

그래프 DB에 저장하여
효율적 검색·관리



Multi-hop Query

복잡한 그래프 기반
추론 쿼리 가능

추출된 관계는 Knowledge Graph 형태로 저장되어 다양한 그래프 기반 추론이 가능

평가 지표

Micro F1

전체 관계에 대한 종합 성능 지표
Precision과 Recall의 조화 평균

Ign F1

학습 데이터에서 본 관계를 제외한
더 엄격한 평가 기준

Evidence F1

근거 문장 예측의 정확도 평가

Inter-sentence F1

문장 간(cross-sentence)
관계 추론 성능 평가

DocRED에서는 단순 정확도보다 F1 기반 평가가 핵심 — Ign F1은 더 엄격한 기준

논문 벤치마크 vs 프로젝트 성능

논문 벤치마크 (DocRED Dev F1)

모델	Dev F1	Dev Ign F1
BERT-RE_base	54.16	—
GAIN-BERT_base	63.49	61.38
ATLOP-BERT_base	64.19	62.23
DREEAM-BERT_base	65.00	63.00

출처: GAIN (Zeng 2020), ATLOP (Zhou 2021)

프로젝트 실험 결과 (Dev F1)

Stage	Dev F1	비고
Stage 1 (Baseline)	__.__ %	
Stage 2 (ATLOP+DREEAM)	__.__ %	최고 성능
Stage 3 (GAIN-lite)	__.__ %	
Stage 4 (Graph U-Net)	__.__ %	

성능 분석 및 아쉬운 점



성능 Gap 원인 분석



학습시간 / epoch 제약



하이퍼파라미터 튜닝 범위 제한



Base 모델만 사용 (large 미사용)



아쉬운 점 / 향후 개선 방향



RoBERTa / DeBERTa encoder 미적용



Graph U-Net pool_ratio 탐색 부족



KG Builer의 실시간 추론 파이프라인 미구현

Stage별 설정 핵심 비교

설정 키	Stage 1	Stage 2	Stage 3	Stage 4
pooling	mean	logsumexp	logsumexp	logsumexp
classifier	bilinear	atlop	atlop	atlop
threshold	fixed	adaptive	adaptive	adaptive
evidence_head	false	true	true	true
graph_encoder	—	—	gain (GCN)	unet
pool_ratio	—	—	—	0.5
loss_type	bce	atlop	atlop	atlop

핵심 기여



End-to-End Pipeline 구축

문서 입력부터 KG 출력까지
일관된 시스템 설계



4-Stage Incremental Stacking 설계

모듈별 점진적 확장으로
체계적 성능 향상



Graph 기반 RE 확장

GAIN-lite GNN + Graph U-Net
구조적 추론 강화 + Ablation



KG 연결

Neo4j 기반
그래프 저장·쿼리

결론



Document-level RE 문제를 4단계 Incremental Stacking으로 체계적으로 해결



Multi-hop reasoning을 GAIN-lite GNN과 Graph U-Net으로 강화



Stage 3 vs Stage 4 Ablation Study를 통한 Graph Encoder 구조 비교



Knowledge Graph로의 확장 가능성 제시 (Neo4j 기반)

참고 논문

논문	내용	적용
Yao et al. (2019) DocRED	데이터셋 구성 및 평가 기준	전체
Devlin et al. (2019) BERT	Document Encoder 기반	전체
Zhou et al. (2021) ATLOP	Adaptive TH + LogSumExp Pooling	Stage 2+
Ma et al. (2023) DREEAM	Evidence-guided Self-training	Stage 2+
Zeng et al. (2020) GAIN	이기종 Entity Graph + GCN	Stage 3
Gao & Ji (2019) Graph U-Net	계층적 TopK 풀링 기반 Graph U-Net	Stage 4

Q & A

감사합니다